Tutorial 1: Markov Decision Processes

Solutions Version

Reinforcement Learning Course

Learning Objectives

By the end of this tutorial, you should be able to:

- Formulate sequential decision problems as MDPs
- Decide whether a state representation satisfies the Markov property
- Compute discounted returns
- Derive the Bellman equation and use it to compute V^π and Q^π exactly for small MDPs
- Relate V^*, Q^* and optimal policies, and recognise the effect of reward design on optimal behaviour

Notation Reference

- $\mathcal{S}$: State space, $s \in \mathcal{S}$: a state
- $\mathcal{A}$: Action space, $a \in \mathcal{A}$: an action
- $P(s'|s,a)$: Transition probability
- $P(s',r|s,a)$: Transition and reward probability
- $R(s,a,s')$: Reward function
- $\pi(a|s)$: Policy (probability of action a in state s)
- $V^\pi(s)$: State value function under policy π
- $Q^\pi(s,a)$: Action value function under policy π
- $V^*(s)$, $Q^*(s,a)$: Optimal state and action value functions
- γ: Discount factor, $0 \leq \gamma < 1$

1 Understanding MDPs

1.1 MDP Formulation

Consider a simple robot navigation problem: A robot is on a 1D track with 5 positions labeled 0, 1, 2, 3, 4. Position 0 is the starting position, and position 4 is the goal (but not terminal). The robot can take actions: `LEFT`, `RIGHT`, or `STAY`.

Let's build the complete MDP formulation $(\mathcal{S}, \mathcal{A}, P, R)$ step by step:

(a) **State and Action Spaces:** Define the state space $\mathcal{S}$ and action space $\mathcal{A}$ for this problem. Be explicit about what each represents.

(b) **Transition Function:** Assume deterministic transitions (the robot always moves as intended). Define the transition function $P(s'|s, a)$ and write it out explicitly for at least 3 state-action pairs. What happens at the boundaries?

(c) **Reward Function:** Design a reward function $R(s, a, s')$ where: reaching the goal gives $+10$, each step costs -1, and staying in place when not at the goal costs -2. Write out $R(s, a, s')$ for at least 3 cases.

Solution

(a) **State and Action Spaces:**

- $\mathcal{S} = \{0, 1, 2, 3, 4\}$ (the five positions on the track)
- $\mathcal{A} = \{\text{LEFT}, \text{RIGHT}, \text{STAY}\}$ (the three available actions)

(b) **Transition Function:**

Since transitions are deterministic, $P(s'|s, a) \in \{0, 1\}$ for all s, a, s'.

Examples:

$$
\begin{aligned}
P(1|2, \text{LEFT}) &= 1.0 \\
P(3|2, \text{RIGHT}) &= 1.0 \\
P(2|2, \text{STAY}) &= 1.0 \\
P(0|0, \text{LEFT}) &= 1.0 \quad \text{(boundary: stay in place)} \\
P(4|4, \text{RIGHT}) &= 1.0 \quad \text{(boundary: stay in place)}
\end{aligned}
$$

Boundary behavior: At positions 0 and 4, attempting to move beyond the boundaries keeps the robot in the same position.

(c) **Reward Function:**

$$
\begin{aligned}
R(3, \text{RIGHT}, 4) &= +10 - 1 = +9 \quad \text{(reaching goal minus step cost)} \\
R(2, \text{RIGHT}, 3) &= -1 \quad \text{(standard step cost)} \\
R(2, \text{STAY}, 2) &= -2 \quad \text{(penalty for staying when not at goal)} \\
R(4, \text{STAY}, 4) &= 0 \quad \text{(at goal, no penalty for staying)} \\
R(1, \text{LEFT}, 0) &= -1 \quad \text{(step cost)} \\
R(0, \text{LEFT}, 0) &= -1 \quad \text{(step cost even at boundary)}
\end{aligned}
$$

1.2 The Markov Property

For each of the following, state whether the given state representation satisfies the Markov property. If it does not, suggest how the state could be augmented so that it (approximately) does.

(a) Pong, where the state is a single frame of the screen.

(b) Pong, where the state is the last 4 frames of the screen stacked together.

(c) A robot arm, where the state is its joint angles only.

(d) A robot arm, where the state is its joint angles and joint velocities.

(e) Texas Hold'em poker, where the state is your own two cards and the cards on the table.

(f) Chess, where the state is the position of the pieces on the board.

Solution

(a) **Not Markov.** A single frame gives the ball's position but not its velocity, so the next frame cannot be predicted from the current one alone.

(b) **(Approximately) Markov.** Stacking frames lets the agent infer the ball's velocity (and acceleration), which is all that is needed to predict the next frame. This is exactly what DQN does on Atari.

(c) **Not Markov.** Knowing the angles alone does not tell you where the arm will be next; you also need to know how fast it is moving.

(d) **(Approximately) Markov.** Angles and velocities together determine the next state given the applied torques (ignoring unmodelled effects such as contact or motor heating).

(e) **Not Markov.** The opponents' cards are hidden, and their past betting behaviour carries information about them. The agent needs a summary of the history (e.g. the betting sequence), or a belief over the hidden cards, to predict the future well.

(f) **Not quite Markov.** The board alone is almost enough, but the rules depend on history: whose turn it is, castling rights, en-passant availability and repetition counts. Adding these to the state makes it Markov.

1.3 Computing Returns

Consider a sequence of rewards: $R_1 = 1$, $R_2 = 2$, $R_3 = 5$, $R_4 = 0$.

(a) Calculate the discounted return G_0 with $\gamma = 0.9$.

(b) Calculate the discounted return G_0 with $\gamma = 0.5$.

(c) What is the return with $\gamma = 0$ (myopic agent)?

(d) Explain intuitively why we use discounting in RL.

(e) Now consider a *continuing* task in which the agent receives a reward of $+1$ at every time step, forever. Calculate G_0 with $\gamma = 0.9$. Give a general expression for a constant reward r and discount γ.

(f) What happens to G_0 in part (e) if $\gamma = 1$? Why is $\gamma = 1$ acceptable for episodic tasks but not for continuing ones?

Solution

(a) With $\gamma = 0.9$:

$$\begin{aligned} G_0 &= R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 \\ &= 1 + 0.9(2) + 0.81(5) + 0.729(0) \\ &= 1 + 1.8 + 4.05 + 0 = 6.85 \end{aligned}$$

(b) With $\gamma = 0.5$:

$$\begin{aligned} G_0 &= 1 + 0.5(2) + 0.25(5) + 0.125(0) \\ &= 1 + 1 + 1.25 + 0 = 3.25 \end{aligned}$$

(c) With $\gamma = 0$ (myopic): $G_0 = 1$ (only immediate reward matters)

(d) **Why use discounting:**

- **Mathematical convenience:** Ensures infinite sums converge
- **Uncertainty about future:** Future rewards are less certain
- **Time preference:** Models preference for sooner rewards
- **Computational tractability:** Limits planning horizon

(e) This is a geometric series:

$$G_0 = \sum_{k=0}^{\infty} \gamma^k r = \frac{r}{1-\gamma} = \frac{1}{1-0.9} = 10$$

(f) With $\gamma = 1$, $G_0 = 1 + 1 + 1 + \cdots = \infty$, so all policies that earn positive reward forever would have infinite return and could not be compared. In an episodic task the episode ends after a finite number of steps, so the (undiscounted) sum is finite and $\gamma = 1$ is well defined.

2 Bellman Equation Derivation

Starting from the definition of the state value function:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$

(a) Show that $G_t = R_{t+1} + \gamma G_{t+1}$.

(b) Derive the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s', r|s, a)[r + \gamma V^\pi(s')]$$

(c) Explain in words what this equation means. Why is it recursive?

Solution

(a) **Show $G_t = R_{t+1} + \gamma G_{t+1}$:**

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

(b) **Derive the Bellman equation:**

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] && \text{(from part a)} \\ &= \mathbb{E}_\pi[R_{t+1}] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s] && \text{(linearity)} \\ &= \mathbb{E}_\pi[R_{t+1}] + \gamma \mathbb{E}_\pi[\mathbb{E}[G_{t+1} | S_{t+1}] | S_t = s] && \text{(law of total expectation)} \\ &= \mathbb{E}_\pi[R_{t+1}] + \gamma \mathbb{E}_\pi[V^\pi(S_{t+1}) | S_t = s] && \text{(recursive definition)} \\ &= \sum_a \pi(a|s) \sum_{s',r} P(s', r | s, a)[r + \gamma V^\pi(s')] && \text{(expand expectations)} \end{aligned}$$

(c) **Intuitive meaning:** The value of a state equals the expected immediate reward plus the discounted value of the next state. It's recursive because $V(s)$ is defined in terms of $V(s')$ for successor states. The future from s consists of an immediate reward and then being in state s', whose value is already defined by the same principle.

3 Solving the Bellman Equation for a Fixed Policy

3.1 Exact Policy Evaluation

Consider a 2-state MDP with states $\{s_1, s_2\}$, actions $\{a_1, a_2\}$ and discount factor $\gamma = 0.9$. The dynamics are:

s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
s_1	a_1	s_1	0.6	4
s_1	a_1	s_2	0.4	1
s_1	a_2	s_2	1.0	3
s_2	a_1	s_1	1.0	1
s_2	a_2	s_1	0.2	-2
s_2	a_2	s_2	0.8	0

The agent follows the deterministic policy $\pi(s_1) = a_1$, $\pi(s_2) = a_2$.

(a) Compute the expected immediate reward $r_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) R(s,a,s')$ for each state, and the state transition matrix P_π under π.

(b) Write down the Bellman equation for $V^\pi(s_1)$ and $V^\pi(s_2)$. This gives two linear equations in two unknowns; solve them exactly.

(c) Write the Bellman equation in matrix form and give the closed-form solution for V^π. Why is this approach impractical for MDPs with many states?

(d) Compute $Q^\pi(s,a)$ for all four state-action pairs, using

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^\pi(s')\right].$$

Check that $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$ holds in each state.

(e) Compare $Q^\pi(s_2, a_1)$ with $V^\pi(s_2)$. What does this suggest about the policy π?

Solution

(a) Under π the agent always takes a_1 in s_1 and a_2 in s_2:

$$r_\pi(s_1) = 0.6 \times 4 + 0.4 \times 1 = 2.8$$
$$r_\pi(s_2) = 0.2 \times (-2) + 0.8 \times 0 = -0.4$$

$$P_\pi = \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix}$$

(b) The Bellman equations are:

$$V^\pi(s_1) = 2.8 + 0.9\,[0.6\,V^\pi(s_1) + 0.4\,V^\pi(s_2)]$$
$$V^\pi(s_2) = -0.4 + 0.9\,[0.2\,V^\pi(s_1) + 0.8\,V^\pi(s_2)]$$

Rearranging:

$$0.46\,V^\pi(s_1) - 0.36\,V^\pi(s_2) = 2.8$$
$$-0.18\,V^\pi(s_1) + 0.28\,V^\pi(s_2) = -0.4$$

The determinant is $0.46 \times 0.28 - 0.36 \times 0.18 = 0.1288 - 0.0648 = 0.064$. By Cramer's rule:

$$V^\pi(s_1) = \frac{2.8 \times 0.28 - 0.36 \times 0.4}{0.064} = \frac{0.784 - 0.144}{0.064} = 10$$
$$V^\pi(s_2) = \frac{0.46 \times (-0.4) + 0.18 \times 2.8}{0.064} = \frac{-0.184 + 0.504}{0.064} = 5$$

Check: $2.8 + 0.9(0.6 \times 10 + 0.4 \times 5) = 2.8 + 7.2 = 10$ ✓, $-0.4 + 0.9(0.2 \times 10 + 0.8 \times 5) = -0.4 + 5.4 = 5$ ✓

(c) In matrix form, $V^\pi = r_\pi + \gamma P_\pi V^\pi$, so

$$V^\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

Inverting an $|\mathcal{S}| \times |\mathcal{S}|$ matrix costs $O(|\mathcal{S}|^3)$, which quickly becomes infeasible as the number of states grows (and requires the full model to be known). This motivates the iterative methods in the next tutorial.

(d)

$$Q^\pi(s_1, a_1) = 0.6(4 + 0.9 \times 10) + 0.4(1 + 0.9 \times 5) = 0.6 \times 13 + 0.4 \times 5.5 = 7.8 + 2.2 = 10$$
$$Q^\pi(s_1, a_2) = 1.0(3 + 0.9 \times 5) = 7.5$$
$$Q^\pi(s_2, a_1) = 1.0(1 + 0.9 \times 10) = 10$$
$$Q^\pi(s_2, a_2) = 0.2(-2 + 0.9 \times 10) + 0.8(0 + 0.9 \times 5) = 0.2 \times 7 + 0.8 \times 4.5 = 1.4 + 3.6 = 5$$

Since π is deterministic, $V^\pi(s) = Q^\pi(s, \pi(s))$: $V^\pi(s_1) = Q^\pi(s_1, a_1) = 10$ ✓ and $V^\pi(s_2) = Q^\pi(s_2, a_2) = 5$ ✓.

(e) $Q^\pi(s_2, a_1) = 10 > V^\pi(s_2) = 5$. Taking a_1 once in s_2 and then following π is better than following π from s_2, so π is **not optimal**: switching to a_1 in s_2 should improve it. This idea (policy improvement) is the subject of the next tutorial.

4 Optimal Value Functions and Policies

4.1 From Q^* to V^* and π^*

You are given the optimal action-value function of an MDP with three states and three actions:

$Q^*(s,a)$	a_1	a_2	a_3
s_1	4.0	6.5	6.5
s_2	3.2	1.0	2.8
s_3	−1.0	−0.5	−2.0

(a) Compute $V^*(s)$ for each state.

(b) Write down an optimal deterministic policy π^*. Is it unique?

(c) Is the stochastic policy that picks a_2 or a_3 with probability 0.5 each in s_1, and acts as in (b) elsewhere, optimal?

(d) Is a policy that always takes a_1 in s_1 (and acts as in (b) elsewhere) optimal? Justify your answer using the table.

Solution

(a) Using $V^*(s) = \max_a Q^*(s,a)$:

$$V^*(s_1) = 6.5, \quad V^*(s_2) = 3.2, \quad V^*(s_3) = -0.5$$

(b) $\pi^*(s) = \arg\max_a Q^*(s,a)$ gives $\pi^*(s_2) = a_1$, $\pi^*(s_3) = a_2$, and $\pi^*(s_1) \in \{a_2, a_3\}$. It is **not unique**: a_2 and a_3 tie in s_1, so there are two optimal deterministic policies. Both achieve the same value function V^*.

(c) **Yes.** Any policy that only puts probability on actions in $\arg\max_a Q^*(s,a)$ is optimal. Here $\sum_a \pi(a|s_1) Q^*(s_1, a) = 0.5 \times 6.5 + 0.5 \times 6.5 = 6.5 = V^*(s_1)$.

(d) **No.** $Q^*(s_1, a_1) = 4.0$ is the best return achievable when taking a_1 in s_1 *even if the agent acts optimally afterwards.* A policy that takes a_1 in s_1 therefore has $V^\pi(s_1) \leq 4.0 < 6.5 = V^*(s_1)$, so it is not optimal.

4.2 Optimal Values and Reward Design

Return to the 1D robot track from Section 1.1, with $\gamma = 0.9$. To remove any ambiguity, the reward function is:

- Moving `RIGHT` from position 3 into position 4: +9 (the +10 goal reward minus the step cost)
- Any other `LEFT` or `RIGHT` action (including bumping into a boundary): -1
- `STAY` at positions 0–3: -2; `STAY` at position 4: 0

(a) First suppose position 4 is a **terminal** state, so $V^*(4) = 0$. Working backwards from the goal, compute $V^*(s)$ for $s = 3, 2, 1, 0$.

(b) Still with position 4 terminal, use $Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$ to compute $Q^*(2,a)$ for all three actions, and state $\pi^*(2)$.

(c) Now suppose, as in the original problem, that position 4 is **not terminal**. Starting from position 3, compute the discounted return of each of the following policies:

(i) Move `RIGHT` into position 4, then `STAY` there forever.

(ii) Move `RIGHT` into position 4, then alternate `LEFT`, `RIGHT`, `LEFT`, `RIGHT`, ... forever.

(d) Which policy in (c) does an optimal agent prefer? What does this tell you about the reward function, and how could you fix it?

Solution

(a) Transitions are deterministic and moving `RIGHT` is clearly best, so $V^*(s) = R(s, \texttt{RIGHT}, s+1) + \gamma V^*(s+1)$:

$$\begin{aligned} V^*(3) &= 9 + 0.9 \times 0 = 9 \\ V^*(2) &= -1 + 0.9 \times 9 = 7.1 \\ V^*(1) &= -1 + 0.9 \times 7.1 = 5.39 \\ V^*(0) &= -1 + 0.9 \times 5.39 = 3.851 \end{aligned}$$

(b)

$$\begin{aligned} Q^*(2, \texttt{LEFT}) &= -1 + 0.9 \times V^*(1) = -1 + 4.851 = 3.851 \\ Q^*(2, \texttt{STAY}) &= -2 + 0.9 \times V^*(2) = -2 + 6.39 = 4.39 \\ Q^*(2, \texttt{RIGHT}) &= -1 + 0.9 \times V^*(3) = -1 + 8.1 = 7.1 \end{aligned}$$

So $\pi^*(2) = \texttt{RIGHT}$, and $V^*(2) = \max_a Q^*(2,a) = 7.1$ ✓.

(c) (i) $G = 9 + 0.9 \times 0 + 0.9^2 \times 0 + \cdots = 9$

(ii) The rewards alternate $+9, -1, +9, -1, \ldots$:

$$\begin{aligned} G &= (9 - 0.9 \times 1) + 0.9^2(9 - 0.9 \times 1) + 0.9^4(9 - 0.9 \times 1) + \cdots \\ &= \frac{9 - 0.9}{1 - 0.81} = \frac{8.1}{0.19} \approx 42.63 \end{aligned}$$

(d) The optimal agent prefers **policy (ii)**: it repeatedly steps out of the goal just to collect the +10 again on re-entry, and never settles at the goal. The reward function rewards *entering* the goal rather than *being at* it, so the agent exploits it. This is a simple example of **reward hacking**: the agent maximises the reward we wrote down, not the behaviour we intended. Possible fixes:

- Make position 4 terminal (as in part (a)), so the goal reward can only be collected once.
- Replace the entry bonus with a reward for *being* at the goal (e.g. $+1$ for every step spent at position 4 and no bonus for entering it), so staying is better than leaving.
- Penalise leaving the goal heavily enough that the round trip is not worth it (here, a penalty L for `LEFT` from position 4 with $L > 8.1$).